Grenoble INP - Ensimag, UGA
Projet GL 2024-2025

Documentation de conception

Groupe 6 Equipe 34

BORNARD Maël, FOKOU DJONTU Manuella, GAUTIER Mattéo,
JEULIN Matteo, MELLOT Arthur

Table des matières :

Table des matières : **2**
1. Introduction **3**
a. Objectif du document 3
b. Contexte 3
2. Architecture générale **4**
a. Vue d'ensemble du compilateur 4
b. Principales étapes de compilation 4
3. Architecture des classes et relation **5**
a. Structure de l'objet DecacCompiler 5
b. Structure du package fr.ensimag.deca.tree 5
c. Génération de labels uniques 6
4. Spécification du code et du compilateur **6**
a. Génération de labels uniques 6
5. Structure de données et algorithmes **7**
a. Pile TSTO 7
6. Test et validation **7**
a. Compilation du projet 7
b. Tests et scripts de tests 7
7. Annexe **10**

1. Introduction

a. Objectif du document

Ce document a pour objectif de présenter la conception du compilateur Déca, développé dans le cadre d'un projet académique à l'Ensimag. Il fournit une vue d'ensemble de l'architecture du compilateur, en mettant en évidence les choix de conception effectués, les algorithmes employés et les justifications techniques qui sous-tendent ces décisions. Ce document s'adresse principalement aux développeurs souhaitant maintenir ou faire évoluer le compilateur en fournissant les informations nécessaires pour comprendre son organisation interne et les choix techniques réalisés.

b. Contexte

Le compilateur Déca a pour objectif de transformer un programme écrit en Déca, un sous-ensemble simplifié du langage Java, en un code assembleur exécutable par la machine virtuelle IMA. Ce projet visait à mettre en pratique les concepts fondamentaux de la compilation, en incluant des étapes classiques telles que l'analyse lexicale, l'analyse syntaxique, l'analyse sémantique, et la génération de code. L'implémentation respecte les spécifications définies par le sujet, en appliquant les techniques et structures de données classiques pour générer un code exécutable.

2. Architecture générale

Le compilateur Deca est structuré en plusieurs composants principaux, chacun responsable d'une étape spécifique du processus de compilation.

a. Vue d'ensemble du compilateur

- **Analyseur lexical (Lexer)** : Convertit le code source en une séquence de jetons (tokens). Cette étape est réalisée par le fichier ANTLR4 `DecaLexer.g4`.
- **Analyseur syntaxique (Parser)** : Analyse la séquence de jetons pour construire un arbre syntaxique abstrait (AST). Cette étape est réalisée par le fichier ANLR4 `DecaParser.g4`.
- **Arbre Syntaxique Abstrait** : Permet de représenter le programme source par ses différents blocs (déclaration, instruction, …) avec leur relations de parenté. Implémenté par les différentes classes du package `fr.ensimag.deca.tree`.
- **Analyse sémantique** : Vérifie la validité sémantique de l'AST, en s'assurant que les règles du langage sont respectées. Cette étape est réalisée par les méthodes de vérification dans les classes de l'arbre syntaxique en plusieurs passes chacune vérifiant certains critères.
- **Générateur de code** : Génère le code assembleur à partir de l'AST vérifié par les différentes classes de l'AST.

b. Principales étapes de compilation

La compilation d'un fichier .deca se fait en plusieurs étapes :

1. **Lexing** : Le fichier source est lu et converti en une séquence de jetons par l'analyseur lexical qui vérifie que le fichier fourni est effectivement constitué de mots du langage deca.
2. **Parsing** : La séquence de jetons est analysée par l'analyseur syntaxique qui s'assure que l'enchaînement des tokens est conforme à la grammaire du langage deca et construit ensuite l'arbre syntaxique abstrait (AST).
3. **Analyse sémantique** : L'AST est vérifié pour s'assurer qu'il respecte les règles sémantiques du langage Deca.
4. **Génération de code** : Le code assembleur est généré dans le bon ordre à partir de l'AST vérifié.

3. Architecture des classes et relation

a. Structure de l'objet `DecacCompiler`

La classe `DecacCompiler` est responsable de la gestion du processus de compilation dans le compilateur Deca. Elle contient des **options de compilation**, le fichier source à compiler, et **l'AST généré par le Parser**. Elle gère également les **environnements de types** et les **tables de symboles** utilisés lors des phases de vérification contextuelles. Les méthodes principales incluent la gestion des erreurs, la génération de code, et l'organisation des différentes phases de compilation.

b. Structure du package `fr.ensimag.deca.tree`

La structure de l'AST est détaillée par le diagramme en notation UML fourni en Annexe.

Chaque classe représente un potentiel nœud de l'AST qui représente notre programme et implémente les différentes méthodes de vérification :

- `verifyListClass`, vérifie la déclaration correcte des classes lors de la première passe de décoration de l'arbre.
- `verifyListClassMembers`, vérifie la déclaration des champs et méthodes de des classes du programme.
- `verifyListClassBody`, vérifie l'initialisation des différents champs et le corps des méthodes (comme `verifyMain`) de toutes les classes.
- `verifyMain`, vérifie les déclarations de variables et les instructions dans la fonction `main`.

Une fois les vérifications effectuées et l'AST décoré la génération du code assembleur se fait en plusieurs étapes par la méthode `codeGenProgram :`

- `genCodeMethodTable` : génère la table des adresses des méthodes pour y accéder facilement lors de l'exécution du programme.
- `codeGenMain` : génère le code pour le Main (déclaration puis instructions).
- `genCodeFieldInit` : génère les méthodes implicites d'initialisation des attributs des classes.
- `genCodeMethodCode` : génère le code qui sera exécuté lors d'un appel de méthode.

Les différentes méthodes qui réalisent ces passes sont implémentées dans les classes en question.

c. Génération de labels uniques

Durant le parcours de l'AST lors de la phase de vérification, chaque élément qui nécessite une définition (comprendre ici les types, attributs, méthodes, variables créées par l'utilisateur) est associé à sa définition qui lui permettra lors de la génération de code d'accéder aux éléments importants le concernant.

ex : Une déclaration de méthode doit accéder à sa classe pour générer son label qui sera utilisé lors des appels de méthodes.
Une variable doit pouvoir accéder à son adresse mémoire lorsqu'on voudra y accéder dans un main ou un appel de méthodes.

Toutes ces informations sont accessibles via la définition associée à un objet. L'implémentation de ces définitions est explicitée par le diagramme UML fourni en Annexe.

4. Spécification du code et du compilateur

a. Génération de labels uniques

La génération de labels uniques dans les structures de contrôle comme `if-then-else` et `while` est importante pour éviter les conflits de noms et assurer un flux de contrôle correct.
Voici comment cela est fait dans le code :
Génération d'un identifiant unique : Un identifiant unique est généré pour chaque instruction nécessitant des labels. Cela garantit que chaque label est unique.

```
UUID uuid = UUID.randomUUID();
String labelId = uuid.toString().replace("-", "");
Label label_else = new Label("else_" + labelId);
Label label_end_if = new Label("end_if_" + labelId);
```

Cette approche permet de s'assurer que les labels utilisés dans les instructions de contrôle sont presque sûrement uniques. En effet la spécification UUID (Universally Unique IDentifier) génère un identifiant hexadécimal de 128 bits (122 en réalité, les 6 derniers étant réservés), ce qui nous assure une unicité suffisante $2^{122} \approx 5.3 \times 10^{23}$. La pile débordera déjà si l'utilisateur utilise autant de boucles dans son code.

5. Structure de données et algorithmes

a. Pile TSTO

La structure de données utilisée pour la gestion des TSTO (Test Stack Overflow) dans la classe `DecacCompiler` est une LinkedList. Cela permet de se rappeler du dernier TSTO lors de la génération du code dans blocs de code imbriqués (boucles).

6. Test et validation

a. Compilation du projet

Maven est un outil de gestion de projet principalement utilisé pour les projets Java. Le fichier `pom.xml` est le cœur du projet. Il contient des informations sur le projet et les configurations utilisées par Maven pour construire le projet et gérer les dépendances du projet. Pour compiler le projet il suffit de taper la commande suivante à la racine du projet.

```
mvn compile
```

b. Tests et scripts de tests

En plus des tests fournis, de nombreux tests deca sont disponibles dans `src/test/` pour les différentes fonctionnalités du compilateur : lexing, parsing, vérification contextuelle et génération de code réparties selon si le code est censé produire une erreur ou non.

Pour les fichiers .deca de test la structure est la suivante :

```
// Description:
//    tests print and println with strings and escape char
//
// [Begin Result]
//Hello World !Hello All !
//This " is a quote.
//This \ is a backslash.
// [End Result]
//
// Historique:
//    cree le 01/01/2025

{
  print("Hello World !");
  println("Hello All !");
  println("This \" is a quote.");
  println("This \\ is a backslash.");
}
```

```
// Description:
//    Test incompatible types in if-then-else branches.
//    Test rule 3.36
//
// [Begin Result]
//invalid_branch_type.deca:16:12: Incompatible types, expected int but got
string. Rule 3.28.
// [End Result]
//
// History:
//    created on 08/01/2025

{
   boolean x = true;
   int y = 42;
   if (x) {
       y = "42"; // Affectation incompatible
   } else {
       y =4;
   }
}
```

Les balises [Begin Result] et [End Result] permettent de comparer le résultat après exécution de ima avec le résultat attendu d’un programme valide ou invalide.

De nombreux scripts bash utilisent ces fichiers de test pour tous les exécuter d'un coup. Ces scripts sont disponible dans `src/test/script`

```
test_all
```

lance tous les scripts de tests : `test_context`, `test_lex`, `test_synt`, `advanced-codegen`.

```
mvn test
```

exécute les tests fournis (`basic_lex`, `basic_synt` …).

Enfin la commande

```
mvn verify
```

compile le projet et exécute tous les tests.

7. Annexe

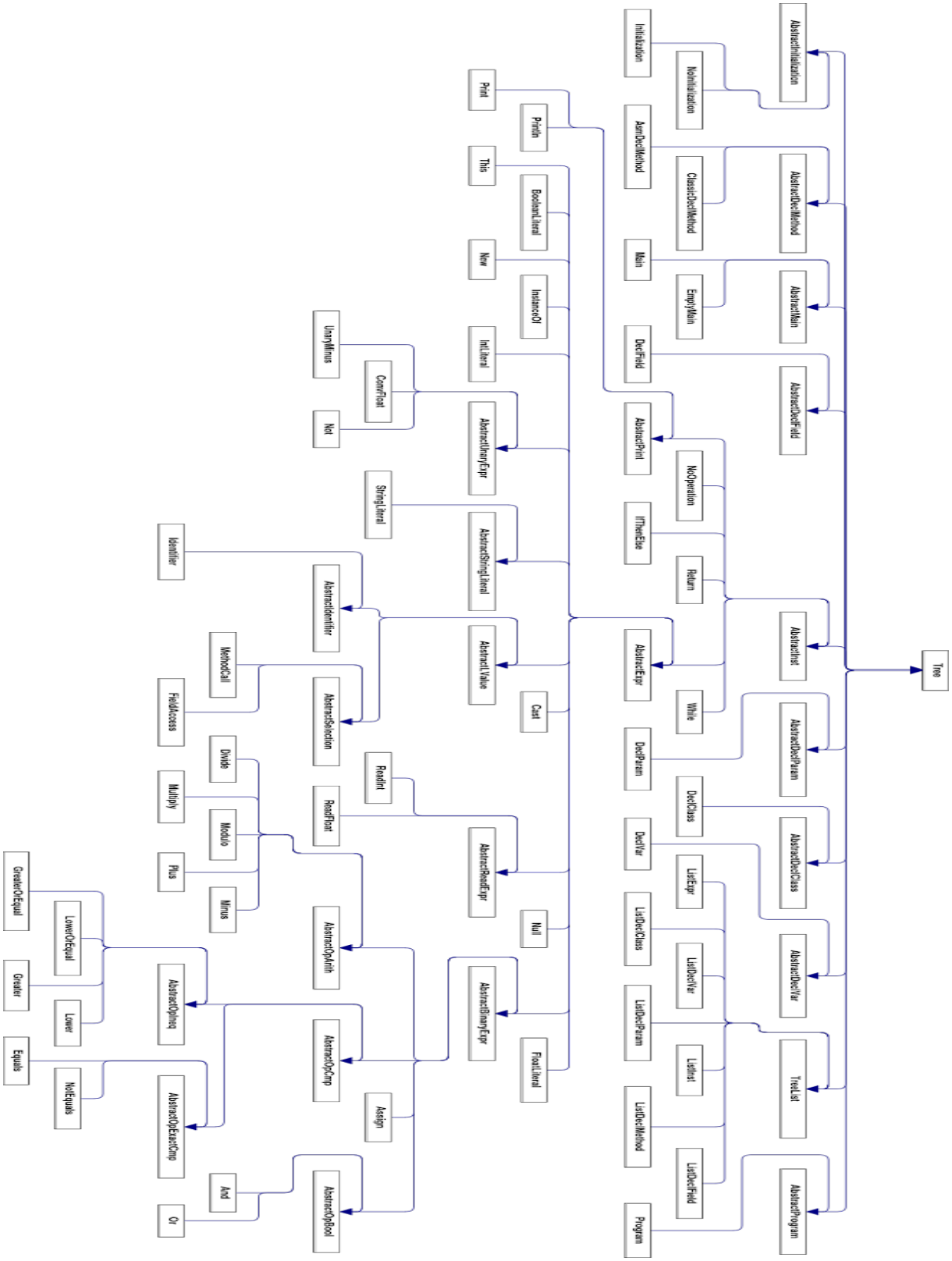

Diagramme UML du package `fr.ensimag.deca.tree` *disponible en format .drawio* [ici]

Groupe 6, équipe 34 : BORNARD Maël, FOKOU DJONTU Manuella, GAUTIER Mattéo, JEULIN Matteo, MELLOT Arthur

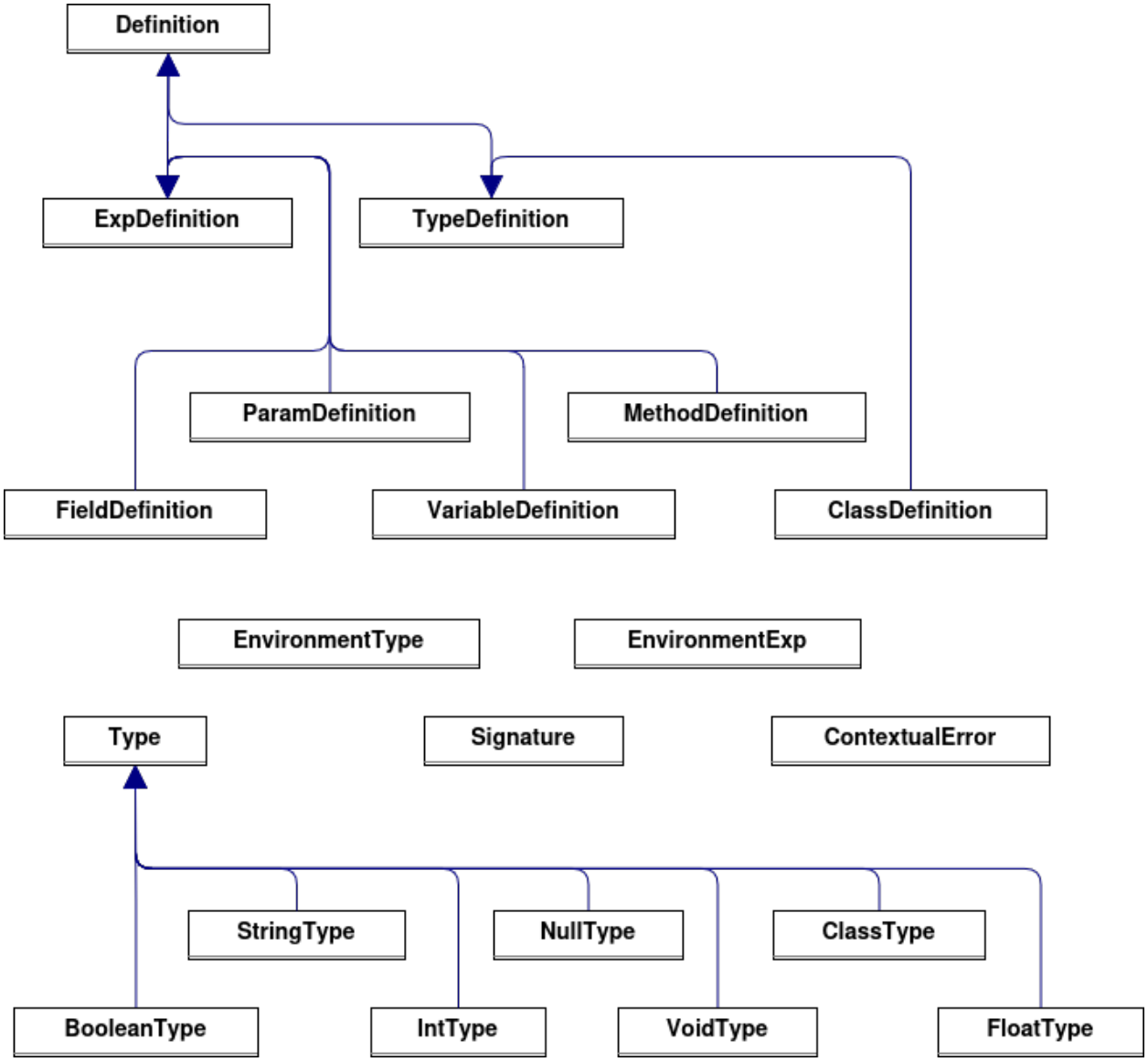

Diagramme UML du package `fr.ensimag.deca.context` *disponible en format .drawio ici*